

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Siti Rafeyfa Asyla.P NIM. 2510817220018

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Siti Rafeyfa Asyla.P
NIM : 2510817220018

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

DAFTAR ISI	3
DAFTAR GAMBAR	5
DAFTAR TABEL	6
SOAL 1: Konversi Adjacency List ke Adjacency Matrix	7
A. Source Code.....	9
B. Output Program.....	11
C. Pembahasan	11
SOAL 2: Konversi Adjacency Matrix ke Adjacency List	13
A. Source Code.....	15
B. Output Program.....	16
C. Pembahasan	17
SOAL 3: BFS: Jarak Terpendek Keluar Labirin	19
A. Source Code.....	21
B. Output Program.....	23
C. Pembahasan	23
SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin	27
A. Source Code.....	29
B. Output Program.....	31
C. Pembahasan	31
SOAL 5: Tree Traversal.....	34
A. Source Code.....	36
B. Output Program.....	39

C. Pembahasan	39
Tautan GIT	42

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	11
Gambar 2. Screenshot Hasil Jawaban Soal 2	16
Gambar 3. Screenshot Hasil Jawaban Soal 3	23
Gambar 4 Screenshot Hasil Jawaban Soal 4	31
Gambar 5. Screenshot Hasil Jawaban Soal 5	39

DAFTAR TABEL

Tabel 1 Source Code Soal 1	9
Tabel 2 Source Code Soal 2	15
Tabel 3 Source Code Soal 3	21
Tabel 4 Source Code Soal 4	29
Tabel 5 Source Code Soal 5	36

SOAL 1: Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi **adjacency matrix**. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .

- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$ • $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A.

Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

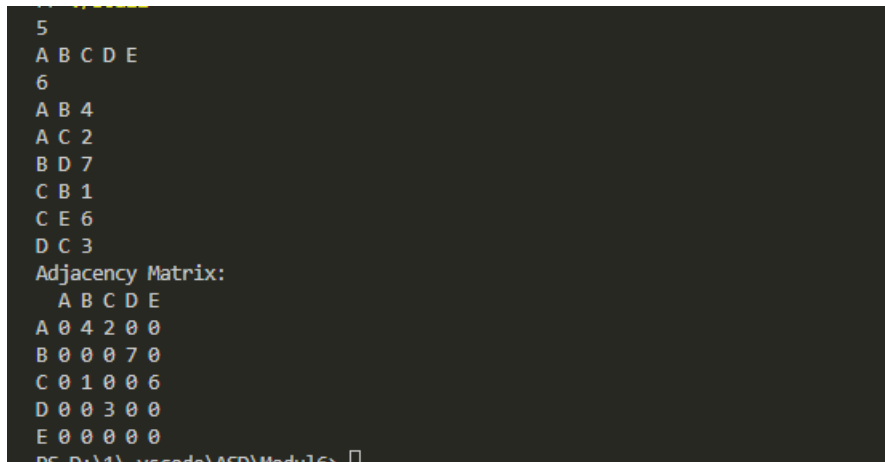
A. Source Code

Tabel 1 Source Code soal1.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>#include <string></code>
4	<code>#include <map></code>
5	
6	<code>int main() {</code>
7	<code> std::ios base::sync_with_stdio(false);</code>
8	<code> std::cin.tie(NULL);</code>
9	
10	<code> int n;</code>
11	<code> if (!(std::cin >> n)) return 0;</code>
12	
13	<code> std::vector<char> vertices(n);</code>
14	<code> std::map<char, int> vertex_to_index;</code>
15	<code> for (int i = 0; i < n; ++i) {</code>
16	<code> std::cin >> vertices[i];</code>
17	<code> vertex_to_index[vertices[i]] = i;</code>
18	<code> }</code>
19	
20	<code> int m;</code>
21	<code> std::cin >> m;</code>
22	

23	<u>std::vector<std::vector<int>></u> matrix(n,
	<u>std::vector<int></u> (n, 0));
24	
25	for (int i = 0; i < m; ++i) {
26	char u, v;
27	int w;
28	<u>std::cin</u> >> u >> v >> w;
29	int idx_u = vertex_to_index[u];
30	int idx_v = vertex_to_index[v];
31	matrix[idx_u][idx_v] = w;
32	}
33	
34	<u>std::cout</u> << "Adjacency Matrix:\n";
35	<u>std::cout</u> << " ";
36	for (int i = 0; i < n; ++i) {
37	<u>std::cout</u> << " " << vertices[i];
38	}
39	<u>std::cout</u> << "\n";
40	
41	for (int i = 0; i < n; ++i) {
42	<u>std::cout</u> << vertices[i];
43	for (int j = 0; j < n; ++j) {
44	<u>std::cout</u> << " " << matrix[i][j];
45	}
46	<u>std::cout</u> << "\n";
47	}
48	
49	return 0;
50	}

B. Output Program



```
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Program dimulai dengan menyertakan pustaka `<iostream>` untuk memfasilitasi operasi masukan dan keluaran standar, `<vector>` sebagai kontainer larik dinamis dalam menyimpan data vertex dan matriks, `<string>` untuk memproses tipe data teks label, serta `<map>` yang berfungsi sebagai tabel pencarian untuk memetakan karakter label vertex ke indeks numerik yang sesuai.

Di dalam fungsi utama `main()`, perintah `std::ios_base::sync_with_stdio(false);` dan `std::cin.tie(NULL);` diterapkan di baris awal untuk menonaktifkan sinkronisasi antara pustaka input-output C dan C++ serta memutus ikatan antara `std::cin` dengan `std::cout`, yang sangat krusial untuk mengoptimalkan kecepatan eksekusi pembacaan data dalam jumlah besar.

Program kemudian mendeklarasikan variabel integer `n` untuk menampung jumlah total vertex, disertai dengan blok pengkondisian `if (!(std::cin >> n)) return 0;` sebagai proteksi guna menghentikan program secara aman jika proses pembacaan nilai `n` mengalami kegagalan atau mencapai akhir berkas input.

Selanjutnya, sebuah objek `std::vector<char> vertices(n);` dialokasikan untuk menyimpan seluruh karakter label vertex sepanjang ukuran `n`, dikombinasikan dengan struktur data `std::map<char, int> vertex_to_index` yang secara berurutan memetakan setiap karakter vertex unik ke dalam indeks integer berbasis 0 (zero-indexed) melalui iterasi perulangan `for` pertama.

Variabel integer `m` kemudian dibaca untuk menentukan jumlah total edge atau sisi berarah yang ada di dalam graph. Struktur penyimpanan data utama diwujudkan melalui deklarasi matriks dua dimensi `std::vector<std::vector<int>> matrix(n, std::vector<int>(n, 0));`, yang menginisialisasi seluruh elemen baris dan kolom berukuran $n \times n$ dengan nilai awal 0 untuk menandakan bahwa pada kondisi awal belum ada edge yang terbentuk antar-vertex.

Proses pengisian adjacency matrix dilakukan di dalam perulangan `for` sepanjang nilai `m`, di mana program membaca representasi edge berarah berupa karakter asal `u`, karakter tujuan `v`, dan bobot integer `w`. Berkat struktur tabel pencarian yang telah dibuat sebelumnya, program dapat langsung mengonversi karakter `u` dan `v` menjadi indeks baris `idx_u` dan indeks kolom `idx_v`, lalu menetapkan nilai bobot `w` secara tepat ke dalam elemen `matrix[idx_u][idx_v]`.

Tahap akhir program adalah mencetak representasi graf ke layar. Baris header matriks dicetak terlebih dahulu dengan menampilkan seluruh label vertex secara horizontal yang dipisahkan oleh spasi. Selanjutnya, melalui struktur perulangan bersarang (*nested loop*), program mencetak label vertex asal di awal setiap baris, diikuti dengan deretan nilai bobot yang tersimpan di dalam matriks untuk setiap kolom yang berkorespondensi, kemudian ditutup dengan karakter *newline* `\n` pada setiap akhir baris sebelum program mengembalikan nilai 0 sebagai tanda eksekusi berhasil.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah **directed weighted graph** yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi **adjacency list**. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency List:

V1 -> (X1,w1), (X2,w2), ...

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 2 Source Code soal2.cpp

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::ios base::sync_with_stdio(false);
6      std::cin.tie(NULL);
7
8      int n;
9      if (!(std::cin >> n)) return 0;
10
11     std::vector<char> vertices(n);
12     for (int i = 0; i < n; ++i) {
13         std::cin >> vertices[i];
14     }
15
16     std::cout << "Adjacency List:\n";
17
18     for (int i = 0; i < n; ++i) {
19         std::cout << vertices[i] << " -> ";
20         bool has_edge = false;
21         bool first = true;
22
23         for (int j = 0; j < n; ++j) {
24             int weight;
25             std::cin >> weight;
```

```

26         if (weight > 0) {
27             if (!first) {
28                 std::cout << ", ";
29             }
30             std::cout << "(" << vertices[j] << "," <<
weight << ")";
31             first = false;
32             has_edge = true;
33         }
34     }
35
36     if (!has_edge) {
37         std::cout << "-";
38     }
39     std::cout << "\n";
40 }
41
42 return 0;
43 }

```

B. Output Program

```

5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -

```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Program dimulai dengan menyertakan pustaka `<iostream>` untuk menangani operasi masukan dan keluaran standar serta `<vector>` sebagai kontainer larik dinamis yang digunakan untuk menyimpan urutan label dari seluruh vertex yang diinputkan.

Di dalam fungsi utama `main()`, perintah `std::ios_base::sync_with_stdio(false);` dan `std::cin.tie(NULL);` dipanggil pada baris awal untuk mematikan sinkronisasi aliran standar C++ dengan C serta melepaskan ikatan `std::cin` dari `std::cout`, sebuah optimasi yang sangat penting untuk mempercepat proses pembacaan data masukan berukuran besar secara signifikan.

Program kemudian mendeklarasikan variabel integer `n` untuk membaca jumlah total vertex dari graf, yang langsung dikawal oleh kondisi proteksi `if (!(std::cin >> n)) return 0;` guna memastikan eksekusi program berhenti secara aman apabila input kosong atau tidak valid.

Sebuah objek larik dinamis `std::vector<char> vertices(n);` dialokasikan dengan ukuran `n` untuk menampung seluruh karakter label vertex. Melalui struktur perulangan `for` pertama, program membaca satu per satu karakter alfabet kapital yang menjadi label unik dari setiap vertex dan menyimpannya ke dalam vektor tersebut secara berurutan berbasis indeks 0.

Setelah seluruh label vertex tersimpan, program mencetak string header `"Adjacency List:\n"`. Proses konversi dari representasi matriks kedekatan ke daftar kedekatan dijalankan langsung secara efisien (*on-the-fly*) tanpa perlu mengalokasikan memori matriks dua dimensi, melainkan menggunakan struktur perulangan bersarang (*nested loop*). Perulangan baris luar (indeks `i`) merepresentasikan vertex asal yang sedang

diproses, di mana program langsung mencetak label vertex tersebut diikuti dengan simbol tanda panah (`vertices[i] << " -> "`).

Di dalam setiap iterasi vertex asal, program menginisialisasi variabel bendera logis `has_edge = false` untuk menandai keberadaan tetangga dan `first = true` untuk mengatur format pemisah koma. Perulangan kolom bagian dalam (indeks `j`) kemudian membaca nilai bobot (`weight`) berarah dari input secara sekuensial. Apabila nilai bobot yang dibaca lebih besar dari 0 (`weight > 0`), hal tersebut menandakan adanya hubungan edge yang valid menuju vertex tujuan ke-`j`.

Jika edge tersebut bukan hubungan pertama yang ditemukan pada baris tersebut (`!first`), program akan mencetak karakter pemisah koma beserta spasi `", "`. Program kemudian mencetak representasi tetangga beserta bobotnya dalam format tanda kurung (`label_tujuan,bobot`). Setelah mencetak, bendera `first` diubah menjadi `false` dan bendera `has_edge` diubah menjadi `true`.

Setelah seluruh nilai bobot pada satu baris selesai diperiksa oleh perulangan dalam, program melakukan pengecekan final lewat kondisi `if (!has_edge)`. Jika bendera tetap bernilai `false` (artinya vertex asal tersebut tidak memiliki sisi keluar sama sekali), program akan mencetak karakter tanda hubung `"-"`. Setiap baris vertex diakhiri dengan pencetakan karakter *newline* `\n` untuk beralih ke baris berikutnya sebelum akhirnya program mengembalikan nilai 0 untuk menandakan eksekusi berjalan dengan sukses dan selesai.

SOAL 3: BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma **Breadth-First Search** (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.

- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0	16

0 1 1 0 0 0 1 0 0 0	
0 0	
7 9	

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3 Source Code soal3.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	<code>#include <queue></code>
4	
5	<code>struct Cell {</code>
6	<code> int r, c, dist;</code>
7	<code>};</code>
8	
9	<code>int main() {</code>
10	<code> std::ios base::sync_with_stdio(false);</code>
11	<code> std::cin.tie(NULL);</code>
12	
13	<code> int r, c;</code>
14	<code> if (!(std::cin >> r >> c)) return 0;</code>
15	
16	<code> std::vector<std::vector<int>>> grid(r,</code>
	<code>std::vector<int>(c));</code>
17	<code> for (int i = 0; i < r; ++i) {</code>

```

18         for (int j = 0; j < c; ++j) {
19             std::cin >> grid[i][j];
20         }
21     }
22
23     int sr, sc, fr, fc;
24     std::cin >> sr >> sc >> fr >> fc;
25
26     std::vector<std::vector<bool>> visited(r,
std::vector<bool>(c, false));
27     std::queue<Cell> q;
28
29     q.push({sr, sc, 0});
30     visited[sr][sc] = true;
31
32     int dr[] = {-1, 1, 0, 0};
33     int dc[] = {0, 0, -1, 1};
34     int result = -1;
35
36     while (!q.empty()) {
37         Cell curr = q.front();
38         q.pop();
39
40         if (curr.r == fr && curr.c == fc) {
41             result = curr.dist;
42             break;
43         }
44
45         for (int i = 0; i < 4; ++i) {
46             int nr = curr.r + dr[i];

```

```

47         int nc = curr.c + dc[i];
48
49         if (nr >= 0 && nr < r && nc >= 0 && nc < c) {
50             if (grid[nr][nc] == 0 && !visited[nr][nc])
51             {
52                 visited[nr][nc] = true;
53                 q.push({nr, nc, curr.dist + 1});
54             }
55         }
56     }
57
58     std::cout << result << "\n";
59     return 0;
60 }

```

B. Output Program

```

8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 3. Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Program dimulai dengan menyertakan pustaka `<iostream>` untuk menangani operasi masukan dan keluaran standar, `<vector>` untuk mengalokasikan kontainer matriks dua dimensi secara dinamis, serta `<queue>` untuk menyediakan struktur data antrean yang menjadi komponen utama dalam menjalankan algoritma Breadth-First Search (BFS).

Sebelum masuk ke fungsi utama, program mendefinisikan sebuah struktur data kustom bernama `struct Cell`. Struktur ini merangkum tiga informasi penting sekaligus, yaitu koordinat baris (`int r`), koordinat kolom (`int c`), serta jarak langkah yang telah ditempuh dari titik awal (`int dist`) untuk setiap sel di dalam labirin.

Di dalam fungsi utama `main()`, perintah `std::ios_base::sync_with_stdio(false);` dan `std::cin.tie(NULL);` dijalankan untuk mengoptimalkan kinerja pembacaan data masukan dengan cara mematikan sinkronisasi aliran standar C++ dan memutuskan ikatan interaksi antara `std::cin` dan `std::cout`.

Program kemudian membaca dimensi labirin melalui variabel baris `r` dan kolom `c`, yang dikawal ketat oleh kondisi `if (!(std::cin >> r >> c)) return 0;` sebagai mekanisme penyelamat untuk menghentikan program jika terjadi kegagalan input data. Setelah dimensi dipastikan valid, matriks dua dimensi `std::vector<std::vector<int>>` `grid(r, std::vector<int>(c))` dialokasikan dan diisi dengan nilai sel berupa angka 0 (jalan) atau 1 (dinding) melalui struktur perulangan bersarang.

Selanjutnya, program membaca empat nilai integer berikutnya yang merepresentasikan titik koordinat awal (`sr, sc`) serta titik koordinat tujuan akhir (`fr, fc`). Untuk mencegah algoritma memproses sel yang sama berulang kali (yang dapat menyebabkan perulangan tak terbatas), sebuah matriks penanda `std::vector<std::vector<bool>>` `visited(r, std::vector<bool>(c, false))` dibuat dengan ukuran yang sama dan diinisialisasi dengan nilai `false`.

Struktur data antrean `std::queue<Cell> q;` dideklarasikan untuk mengatur urutan penjelajahan sel berbasis level (jarak terdekat). Langkah awal BFS dimulai dengan memasukkan koordinat start ke dalam antrean lewat perintah `q.push({sr,`

`sc, 0}));` dengan jarak mula-mula bernilai 0, sekaligus mengubah status koordinat tersebut pada matriks penanda menjadi `visited[sr][sc] = true;`.

Untuk memfasilitasi pergerakan ke empat arah mata angin (atas, bawah, kiri, kanan), program menyiapkan dua larik konstan perubahan indeks, yaitu `int dr[] = {-1, 1, 0, 0}` dan `int dc[] = {0, 0, -1, 1}`. Sebuah variabel `result` juga diinisialisasi dengan nilai default -1 untuk menampung hasil akhir seandainya titik tujuan tidak dapat dicapai sama sekali.

Proses inti BFS dijalankan di dalam perulangan `while (!q.empty())`. Pada setiap iterasi, program mengambil elemen sel terdepan dalam antrean menggunakan `q.front()`, menyimpannya ke dalam variabel `curr`, dan langsung menghapusnya dari antrean menggunakan `q.pop()`. Evaluasi pertama yang dilakukan adalah memeriksa apakah koordinat `curr` saat ini sudah sama dengan koordinat tujuan (`fr, fc`). Jika kondisi tersebut terpenuhi, variabel `result` akan diisi dengan nilai `curr.dist` dan perulangan segera dihentikan paksa menggunakan perintah `break`.

Jika belum mencapai tujuan, program akan mengeksplorasi empat arah tetangga menggunakan perulangan `for` sepanjang 4 kali. Koordinat baru tetangga dikalkulasikan melalui rumus `nr = curr.r + dr[i]` dan `nc = curr.c + dc[i]`. Program kemudian melakukan validasi berlapis: pertama, memastikan koordinat baru berada di dalam batas grid (`nr >= 0 && nr < r && nc >= 0 && nc < c`); kedua, memastikan sel tersebut merupakan jalan yang dapat dilewati (`grid[nr][nc] == 0`); dan ketiga, memastikan sel tersebut belum pernah dikunjungi sebelumnya (`!visited[nr][nc]`).

Apabila ketiga syarat validasi tersebut terpenuhi, sel tetangga tersebut langsung ditandai dengan mengubah nilai `visited[nr][nc] = true;` untuk mengunci statusnya, lalu dimasukkan ke dalam antrean melalui perintah `q.push({nr, nc, curr.dist + 1})` dengan membawa nilai jarak yang telah ditambah 1 langkah

dari sel sebelumnya. Proses ini berulang secara sistematis hingga antrean kosong atau jalan keluar ditemukan.

Pada bagian akhir, program mencetak nilai yang tersimpan di dalam variabel `result` ke layar, diikuti dengan karakter *newline* `\n`, kemudian mengembalikan nilai 0 untuk menandakan bahwa seluruh rangkaian instruksi program telah selesai dieksekusi dengan sukses.

SOAL 4: DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma **Depth-First Search** (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses **backtracking** agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.

- (FR, FC) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5). Dengan

DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4 Source Code soal4.cpp

1	<code>#include <iostream></code>
2	<code>#include <vector></code>
3	
4	<code>int r, c;</code>
5	<code>std::vector<std::vector<int>> grid;</code>
6	<code>std::vector<std::vector<bool>> visited;</code>
7	<code>int fr, fc;</code>
8	<code>int path_count = 0;</code>
9	
10	<code>int dr[] = {-1, 1, 0, 0};</code>
11	<code>int dc[] = {0, 0, -1, 1};</code>
12	
13	<code>void dfs(int cr, int cc) {</code>
14	<code> if (cr == fr && cc == fc) {</code>
15	<code> path_count++;</code>
16	<code> return;</code>
17	<code> }</code>
18	
19	<code> for (int i = 0; i < 4; ++i) {</code>
20	<code> int nr = cr + dr[i];</code>
21	<code> int nc = cc + dc[i];</code>

```

22
23         if (nr >= 0 && nr < r && nc >= 0 && nc < c) {
24             if (grid[nr][nc] == 0 && !visited[nr][nc]) {
25                 visited[nr][nc] = true;
26                 dfs(nr, nc);
27                 visited[nr][nc] = false;
28             }
29         }
30     }
31 }
32
33 int main() {
34     std::ios base::sync_with_stdio(false);
35     std::cin.tie(NULL);
36
37     if (!(std::cin >> r >> c)) return 0;
38
39     grid.assign(r, std::vector<int>(c));
40     for (int i = 0; i < r; ++i) {
41         for (int j = 0; j < c; ++j) {
42             std::cin >> grid[i][j];
43         }
44     }
45
46     int sr, sc;
47     std::cin >> sr >> sc >> fr >> fc;
48
49     visited.assign(r, std::vector<bool>(c, false));
50     visited[sr][sc] = true;
51

```

52	dfs(sr, sc);
53	
54	std::cout << path_count << "\n";
55	return 0;
56	}

B. Output Program

```

5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Program dimulai dengan menyertakan pustaka `<iostream>` untuk memfasilitasi operasi masukan dan keluaran standar serta `<vector>` untuk mengalokasikan kontainer matriks dua dimensi secara dinamis. Berbeda dengan program sebelumnya, variabel-variabel utama seperti dimensi labirin (`r` dan `c`), matriks labirin (`grid`), matriks penanda (`visited`), koordinat tujuan (`fr` dan `fc`), serta akumulator jumlah jalur (`path_count = 0`) dideklarasikan sebagai variabel global agar dapat diakses secara langsung oleh fungsi rekursif tanpa perlu dilewatkan sebagai argumen. Program juga menyiapkan dua larik konstan global, yaitu `int dr[] = {-1, 1, 0, 0}` dan `int dc[] = {0, 0, -1, 1}` untuk memfasilitasi pergerakan ke empat arah mata angin.

Logika utama pencarian jalur diimplementasikan ke dalam fungsi rekursif `void dfs(int cr, int cc)` yang menerima parameter koordinat sel saat ini (`cr`, `cc`). Basis dari rekursi ini berada pada kondisi `if (cr == fr && cc == fc)`. Ketika koordinat sel saat ini telah mencapai koordinat tujuan akhir, nilai variabel `path_count` akan bertambah satu langkah (inkremen) untuk menandakan

ditemukannya satu jalur sederhana yang valid, lalu fungsi segera berhenti dan kembali (`return`) ke tumpukan panggilan sebelumnya.

Jika belum mencapai tujuan, fungsi `dfs` akan mengeksplorasi tetangga di sekitarnya menggunakan perulangan `for` sebanyak 4 kali. Koordinat baru dihitung melalui rumus `nr = cr + dr[i]` dan `nc = cc + dc[i]`. Validasi pertama dilakukan melalui kondisi `if (nr >= 0 && nr < r && nc >= 0 && nc < c)` untuk memastikan bahwa koordinat baru tidak keluar dari batas wilayah grid labirin.

Validasi kedua memeriksa kelayakan sel melalui kondisi `if (grid[nr][nc] == 0 && !visited[nr][nc])`. Jika sel tersebut merupakan jalan yang dapat dilewati dan belum pernah dikunjungi dalam jalur yang sama, program akan menjalankan mekanisme *backtracking*. Langkah pertama adalah mengunci sel tersebut dengan perintah `visited[nr][nc] = true;`, kemudian program melakukan pemanggilan rekursif ke tingkat yang lebih dalam lewat perintah `dfs(nr, nc);` untuk melanjutkan pencarian dari sel baru tersebut. Setelah penjelajahan dari cabang rekursi tersebut selesai (baik karena menemui jalan buntu maupun berhasil mencapai tujuan), program membuka kembali kunci sel tersebut dengan perintah `visited[nr][nc] = false;` agar sel yang sama dapat digunakan kembali oleh variasi kombinasi jalur alternatif lainnya.

Di dalam fungsi utama `main()`, optimasi kecepatan eksekusi aliran data diterapkan melalui perintah `std::ios_base::sync_with_stdio(false);` dan `std::cin.tie(NULL);`. Program kemudian membaca nilai dimensi baris `r` dan kolom `c`. Jika proses pembacaan gagal, kondisi `if (!(std::cin >> r >> c))` akan langsung menghentikan program demi keamanan data.

Setelah dimensi dipastikan valid, alokasi memori untuk variabel global dilakukan menggunakan fungsi `.assign()`, di mana `grid.assign(r, std::vector<int>(c));` membentuk matriks labirin yang kemudian diisi

nilainya (0 atau 1) melalui perulangan bersarang. Program selanjutnya membaca koordinat titik awal (*sr*, *sc*) serta koordinat titik akhir tujuan (*fr*, *fc*). Matriks penanda juga dialokasikan lewat perintah `visited.assign(r, std::vector<bool>(c, false));` untuk menginisialisasi seluruh status kunjungan menjadi `false`, dan langsung menandai titik awal perjalanan dengan perintah `visited[sr][sc] = true;`.

Proses pencarian keseluruhan jalur dipicu melalui pemanggilan fungsi `dfs(sr, sc)`; dari titik awal. Setelah seluruh kemungkinan kombinasi jalur selesai ditelusuri secara mendalam oleh algoritma DFS dan *backtracking*, program mencetak nilai total jalur yang terkumpul di dalam variabel `path_count` ke layar, diikuti dengan karakter *newline* `\n`, lalu mengembalikan nilai 0 untuk menandakan bahwa program telah berhasil menyelesaikan seluruh instruksi dengan sempurna.

SOAL 5: Tree Traversal

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah **Red Black Tree** secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- **PREORDER**, yaitu urutan root, subtree kiri, lalu subtree kanan.
- **INORDER**, yaitu urutan subtree kiri, root, lalu subtree kanan.
- **POSTORDER**, yaitu urutan subtree kiri, subtree kanan, lalu root.
- **ALL**, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.

- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$

- $-10000000000 \leq A_i \leq 10000000000$

- $1 \leq Q \leq 20$

- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70
50 30 70 20 40 60 80	60 80
4	[Inorder] : 20 30 40 50 60
PREORDER	70 80
INORDER	[Postorder] : 20 40 30 60 80
POSTORDER	70 50
ALL	[Preorder] : 50 30 20 40 70
	60 80
	[Inorder] : 20 30 40 50 60
	70 80 [Postorder] : 20 40 30
	60 80 70 50

Penjelasan Contoh

Bilangan dimasukkan ke RBTREE secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTREE selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTREE yang dibentuk sudah konsisten dengan aturan RBTREE.

A. Source Code

Tabel 5 Source Code soal5.cpp

```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include "RedBlackTree.h"
5
6  void printPreorder(const RedBlackTree::Node* node, const
   RedBlackTree::Node* nil) {
7      if (node == nil) return;
8      std::cout << " " << node->key;
9      printPreorder(node->left, nil);
10     printPreorder(node->right, nil);
11 }
12
13 void printInorder(const RedBlackTree::Node* node, const
   RedBlackTree::Node* nil) {
14     if (node == nil) return;
15     printInorder(node->left, nil);
16     std::cout << " " << node->key;
17     printInorder(node->right, nil);
18 }
19
```

```

20 void printPostorder(const RedBlackTree::Node* node, const
   RedBlackTree::Node* nil) {
21     if (node == nil) return;
22     printPostorder(node->left, nil);
23     printPostorder(node->right, nil);
24     std::cout << " " << node->key;
25 }
26
27 void handleQuery(const std::string& query, const
   RedBlackTree& tree) {
28     if (tree.empty()) {
29         std::cout << "Tree kosong. Tidak ada yang bisa
   ditampilkan.\n";
30         return;
31     }
32
33     const RedBlackTree::Node* root = tree.root();
34     const RedBlackTree::Node* nil = tree.nil();
35
36     if (query == "PREORDER") {
37         std::cout << "[Preorder]   :";
38         printPreorder(root, nil);
39         std::cout << "\n";
40     } else if (query == "INORDER") {
41         std::cout << "[Inorder]     :";
42         printInorder(root, nil);
43         std::cout << "\n";
44     } else if (query == "POSTORDER") {
45         std::cout << "[Postorder]  :";
46         printPostorder(root, nil);

```

```

47         std::cout << "\n";
48     } else if (query == "ALL") {
49         std::cout << "[Preorder]  :";
50         printPreorder(root, nil);
51         std::cout << "\n";
52
53         std::cout << "[Inorder]   :";
54         printInorder(root, nil);
55         std::cout << "\n";
56
57         std::cout << "[Postorder] :";
58         printPostorder(root, nil);
59         std::cout << "\n";
60     }
61 }
62
63 int main() {
64     std::ios base::sync_with_stdio(false);
65     std::cin.tie(NULL);
66
67     int n;
68     if (!(std::cin >> n)) return 0;
69
70     RedBlackTree tree;
71     for (int i = 0; i < n; ++i) {
72         int val;
73         std::cin >> val;
74         if (!tree.contains(val)) {
75             tree.insert(val);
76         }

```

```

77     }
78
79     int q;
80     if (!(std::cin >> q)) return 0;
81
82     for (int i = 0; i < q; ++i) {
83         std::string query;
84         std::cin >> query;
85         handleQuery(query, tree);
86     }
87
88     return 0;
89 }

```

B. Output Program

```

7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder]   : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder]   : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Program dimulai dengan menyertakan pustaka `<iostream>` untuk memfasilitasi operasi masukan dan keluaran standar, `<vector>` untuk menyediakan fungsi penampung data dinamis, `<string>` untuk memproses teks string pada kueri, serta berkas header kustom "RedBlackTree.h" yang mendefinisikan struktur, relasi pointer, dan karakteristik dari struktur data pohon biner Red-Black Tree.

Program mendefinisikan tiga fungsi rekursif terpisah untuk melakukan penelusuran struktur pohon (*tree traversal*). Karena Red-Black Tree pada kode ini menggunakan node penjaga (sentinel node) sebagai representasi daun kosong, basis dari ketiga fungsi rekursif tersebut diperiksa melalui kondisi `if (node == nil) return;`. Fungsi `printPreorder` mengeksekusi penelusuran dengan urutan mencetak kunci node saat ini (`root`), lalu melangkah ke cabang kiri, dan dilanjutkan ke cabang kanan. Fungsi `printInorder` menjalankan penelusuran dari anak cabang kiri paling ujung, mencetak node saat ini, lalu beralih ke anak cabang kanan. Sementara fungsi `printPostorder` menelusuri anak cabang kiri dan kanan secara mendalam terlebih dahulu sebelum mencetak nilai dari node induknya.

Proses penanganan perintah kueri diatur di dalam fungsi `void handleQuery(const std::string& query, const RedBlackTree& tree)`. Fungsi ini mengevaluasi status awal pohon menggunakan kondisi `if (tree.empty())`. Jika terbukti belum ada elemen data yang tersimpan, program akan langsung mencetak pesan kesalahan "Tree kosong. Tidak ada yang bisa ditampilkan.\n" dan segera keluar dari fungsi (`return`).

Jika kondisi pohon berisi data, fungsi `handleQuery` mengambil alamat memori akar pohon (`tree.root()`) dan node pembatas (`tree.nil()`). Melalui struktur pengkondisian `if-else` bertingkat, fungsi mencocokkan string kueri yang diterima dengan empat instruksi yang valid, yaitu "PREORDER", "INORDER", "POSTORDER", atau "ALL". Apabila kueri bernilai "ALL", program akan mengeksekusi ketiga fungsi traversal secara berurutan mulai dari Preorder, Inorder, hingga Postorder lengkap dengan format judul di awal baris.

Di dalam fungsi utama `main()`, optimasi performa pembacaan data diatur di baris awal menggunakan perintah `std::ios_base::sync_with_stdio(false);` dan `std::cin.tie(NULL);`. Program kemudian membaca variabel `n` sebagai penanda jumlah elemen angka yang akan dimasukkan ke dalam pohon. Kondisi `if`

`(!(std::cin >> n))` dipasang untuk menghentikan program dengan aman jika input data kosong.

Setelah objek `RedBlackTree tree;` dideklarasikan, program menjalankan perulangan `for` sebanyak `n` kali untuk membaca nilai integer `val`. Guna memenuhi aturan spesifikasi soal agar tidak ada elemen ganda yang tersimpan, program melakukan penyaringan ketat melalui kondisi `if (!tree.contains(val))`. Fungsi pencarian `.contains(val)` akan memeriksa keberadaan angka tersebut di dalam pohon; jika angka belum terdaftar, barulah perintah `.insert(val)` dipanggil untuk memasukkan angka tersebut dan menyeimbangkan warna serta rotasi node secara otomatis.

Tahap akhir dari fungsi `main()` adalah membaca jumlah kueri lewat variabel `q`. Melalui perulangan `for` sepanjang nilai `q`, program membaca string kueri secara sekuensial dari media masukan dan langsung melemparkannya ke fungsi `handleQuery(query, tree)` untuk diproses hingga selesai. Program kemudian mengakhiri eksekusi dengan mengembalikan nilai 0 untuk menandakan seluruh alur logika telah sukses dijalankan tanpa error.

Tautan GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-srafeyfaa71.git>